Retail Media Insights Platform: Solution Design

Summary

The platform collects impressions, clicks and add-to-cart events from many retailer websites and turns them into campaign insights. Brands and retailers read them through three APIs: within seconds for today (preliminary), and exactly once each day is reconciled (final).

- **Kafka** carries every event, keyed by tenant:customer, so each customer's events stay together and in order.
- **Kafka Streams** deduplicates, enriches and joins clicks to later add-to-cart events, with local state backed up in Kafka.
- **ClickHouse** serves the APIs from pre-aggregated, mergeable unique counts.
- **S3 and Spark** keep the raw history and rebuild each finished day exactly every night.
- **One NGINX edge** serves events, the portal and the APIs; a **CDN** delivers ads to customers from the nearest edge.
- **tenant_id** is in every key, row and file, and drives isolation, quotas and per-tenant configuration.

Assumptions

- **Tenant** = retailer (Tesco, Amazon, Walmart); **sub-tenant** = a banner or business unit (Tesco Groceries). Brands such as Nestlé are users inside a retailer's tenant and see only their own campaigns.
- **Metrics count unique customers**, not events. Identity is cust_id when logged in; otherwise, cust_anonymous_id from a first-party cookie; a login links the two.
- **clickToBasket**: customers who clicked the ad, then added a campaign product to the cart within the attribution window. Model and window are set per tenant (default last click, 7 days).
- **Billing** is invoiced monthly from reconciled numbers only. Real-time numbers drive dashboards and budget pacing.

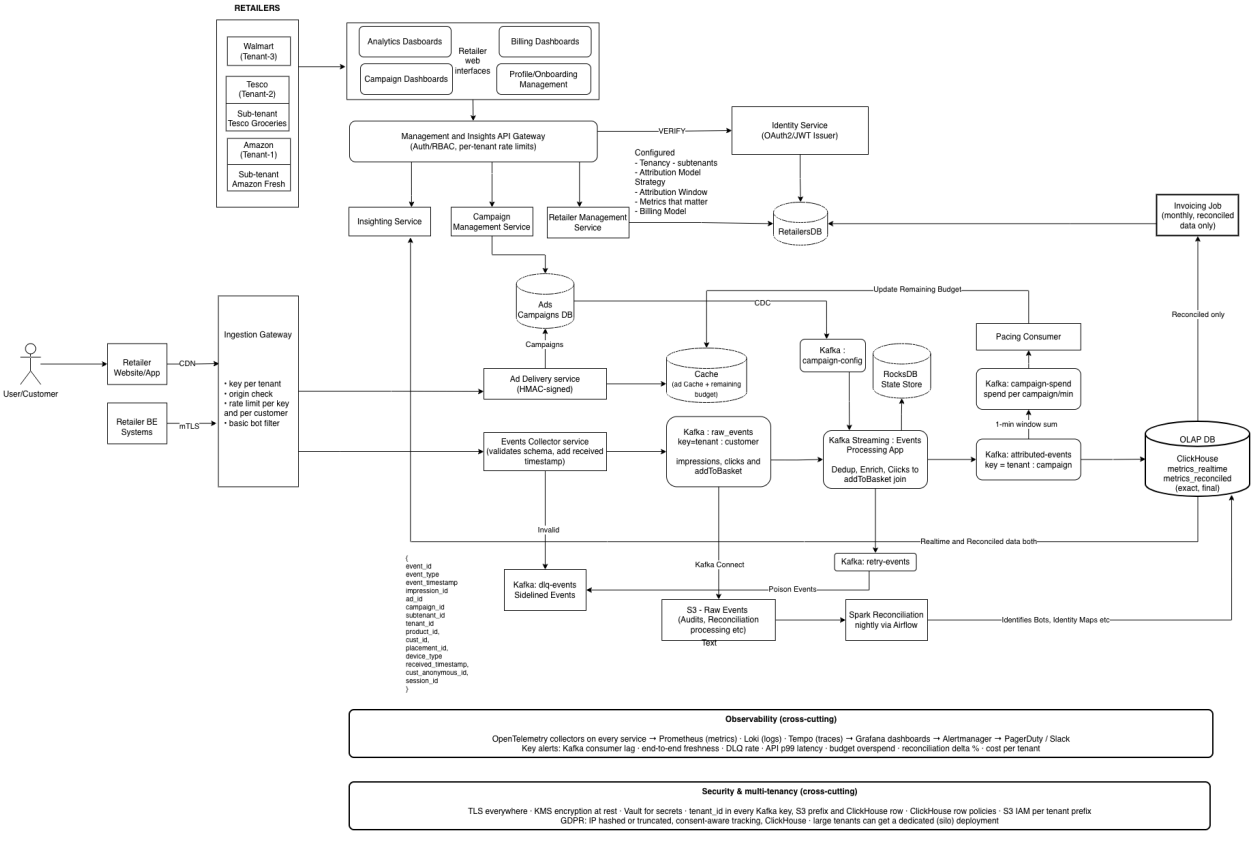
Architecture

The high-level design of the platform is divided into five areas:

- **Retailer portal and management:**
  - React dashboards behind the Management and Insights API Gateway
  - Identity Service issues JWTs that the Insighting, Campaign and Retailer Management services verify
  - RetailersDB holds per-tenant settings (sub-tenants, attribution, metrics, billing)
  - Ads Campaigns DB holds campaigns and budgets.

- **Ingestion:**
    - Browsers and retailer backends (over mTLS) send events to the Ingestion Gateway (write key per tenant, origin check, rate limits, bot filter)
    - Events Collector produces them to Kafka.

- **Ad serving and pacing:**
    - Ad Delivery service serves HMAC-signed ads within the cached remaining budget, which the Pacing Consumer keeps up to date.
- **Streaming:**
    - Kafka Streams turns raw_events into attributed-events, with retry and dead-letter (DLQ) topics
    - Campaign configuration arrives by CDC.
- **Storage and batch:**
    - ClickHouse serves real-time and reconciled metrics
    - Raw events are archived to S3 and reconciled nightly by Spark
    - Monthly invoicing reads reconciled data only.



End-to-end data flow

1. **Ad served.** The page asks the Ad Delivery service for an ad, which checks the campaign's remaining budget and returns it with a signed impression token. The CDN delivers the ad to the customer.
2. **Events sent.** The browser sends impressions and clicks in batches with the tenant's write key; the retailer back end sends add-to-cart events over mTLS.
3. **Collected.** The Events Collector validates the schema and token and adds received_timestamp. Valid events go to raw_events; invalid ones go to dlq-events.
4. **Processed.** Kafka Streams deduplicates, enriches, and joins clicks to later add-to-cart events, then writes to attributed-events (key **tenant:campaign**), which a sink connector loads into metrics_realtime.
5. **Paced.** Spend is summed per campaign per minute into campaign-spend. The Pacing Consumer updates the remaining budget, so an exhausted campaign stops being served.
6. **Archived and reconciled.** Kafka Connect copies raw events to S3. Each night, Spark, scheduled by Airflow, rebuilds the previous day in metrics_reconciled.
7. **Queried and invoiced.** The Insights API returns final numbers for closed days and preliminary numbers for today, each labelled. Invoicing reads reconciled data only.

Event model and partitioning

All event types share one schema; the source sets some fields, and the rest are added on the way in.

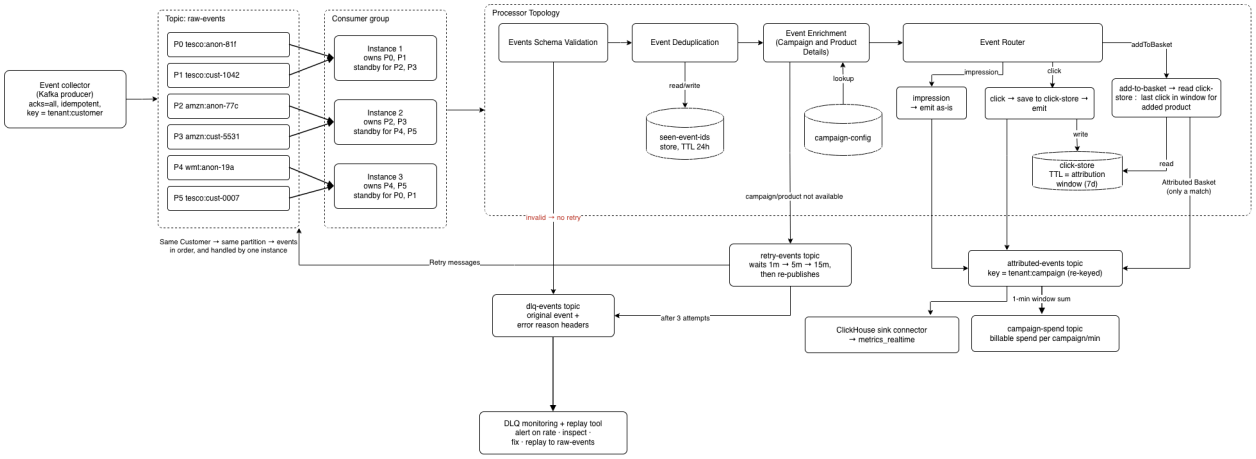
Field	Set by	Purpose
event_id	Source, reused on retries	Deduplication
event_type, event_timestamp	Source	Type; event time for windows and attribution
received_timestamp	Events Collector	Lateness and freshness
tenant_id, subtenant_id	Gateway, from the write key	Isolation, quotas, configuration
cust_id, cust_anonymous_id, session_id	Source	Unique customers, sessions, joins
impression_id, ad_id, campaign_id, placement_id	Ad Delivery token	Links a click to its ad
product_id, device_type	Source; gateway (user agent)	Product scope matching; reporting

Tech Stack:

- Kafka absorbs very high write rates, keeps events for replay and lets several consumers read them independently.
- Kafka Streams is a library, so stream processing deploys like any service with no extra cluster.
- ClickHouse aggregates billions of rows in under a second and can merge unique counts.
- S3 with Parquet is the cheapest durable history, readable by any batch engine.
- PostgreSQL suits the small, transactional configuration data, and Redis gives microsecond budget checks.

Partitioning. Kafka keeps order only within a partition, and a stream instance can only join events it receives. So raw_events is keyed by tenant:customer: all of one customer's events land on the same partition, in order. Customer IDs have huge cardinality, so load spreads evenly; bots, the one-hot-key risk, are rate-limited at the gateway. After the join, results are re-keyed to tenant:campaign, because the APIs count per campaign. The Collector produces with acks=all and idempotence, so an acknowledged event is never lost or duplicated.

Stream processing and attribution



Each consumer-group instance owns some partitions and keeps a warm standby of another instance's state. Inside each instance, the topology runs:

1. **Validate** schema and impression token; invalid events go to dlq-events with no retry.
2. **Deduplicate** against event IDs seen in the last 24 hours.
3. **Enrich** with campaign and product details from campaign-config, a GlobalKTable fed by CDC. If the campaign isn't there yet, the event goes to retry-events.
4. **Route.** Impressions pass through; clicks are saved to the click store and passed through; an add-to-basket reads the customer's last click in the window for that product and emits an attributed basket.

5. **Aggregate** billable spend per campaign per minute into campaign-spend.

Local state (RocksDB)	Holds	Kept for
seen-event-ids	Event IDs already processed	24 hours
click-store	Each customer's recent clicks: campaign, ad, time	Attribution window (7 days)
campaign-config	Full copy of campaign scope and settings	Always; updated by CDC

Fault tolerance. Every store change is also written to a Kafka changelog topic, so a standby takes over in seconds or a new instance rebuilds the store. With exactly_once_v2, reading, updating state, writing and committing offsets are one transaction, so a crash never double counts.

Late events and failures. Processing uses event time, with a 10-minute grace period for an add-to-cart that arrives before its click. Retries (1, 5, 15 minutes) and the DLQ keep one bad event from blocking a partition; see the failure scenarios below.

Storage and Retention

Store	Technology	Holds	Retention
Event log	Kafka	All raw and processed events; replayable	7 days
Raw data lake	S3, Parquet, by tenant and date	Every raw event, once	90 days, then Glacier to 13 months
Insights	ClickHouse	Hourly unique-count states per campaign, ad, placement	Real-time 3 days; reconciled 25 months
Stream state	RocksDB + changelog	Click store, seen IDs, campaign config	Per attribution window
Configuration	PostgreSQL	Tenants, campaigns, budgets	Indefinite
Budget cache	Redis (or Aerospike)	Ads and remaining budget	Live values only

Real-time vs reconciled. metrics_realtime is written by the stream within seconds. metrics_reconciled is rebuilt from S3 each night without bots and duplicates, with late events

and cross-device matches. Both store the same states, so a range spanning both merges into one exact total. Spark swaps in a corrected day with REPLACE PARTITION.

Analytics and privacy. Query the S3 lake directly with Spark Jobs. Right-to-be-forgotten requests are applied by rewriting the affected S3 and ClickHouse partitions; IP addresses are hashed or truncated before storage.

Insights API

Endpoint	**Returns unique customers who…**
GET /ad/{campaignID}/impressions	saw the campaign's ads
GET /ad/{campaignID}/clicks	clicked them
GET /ad/{campaignID}/clickToBasket	added a campaign product to the cart after clicking, within the window

Optional from and to dates (default the last 7 days). The service splits the range at the reconciliation watermark: closed days come from metrics_reconciled, open days from metrics_realtime, merged into one total and labelled per day:

```
{
 "campaignId": "c-4821",
 "metric": "clicks",
 "from": "2026-10-01",
 "to": "2026-10-19",
 "uniqueCustomers": 184230,
 "reconciledThrough": "2026-10-18",
 "breakdown": [
  {
   "date": "2026-10-18",
   "uniqueCustomers": 9600,
   "status": "final",
   "invalidRemoved": 400
  },
  {
   "date": "2026-10-19",
   "uniqueCustomers": 4120,
   "status": "preliminary"
  }
 ]
}
```

- **Fast:** queries read hourly aggregates, so six months is thousands of rows, not billions. Closed days are cached for an hour; today for 30 seconds.
- **Safe:** the tenant comes only from the verified JWT. The service checks the campaign belongs to the caller's tenant (and brand) and otherwise returns 404, so it never reveals that the campaign exists.
- **Resilient:** ClickHouse replicas, timeouts and circuit breakers; per-client limits at NGINX and per-tenant limits in the service.

Deployment overview

Assumption: Availability of infrastructure on AWS Cloud.

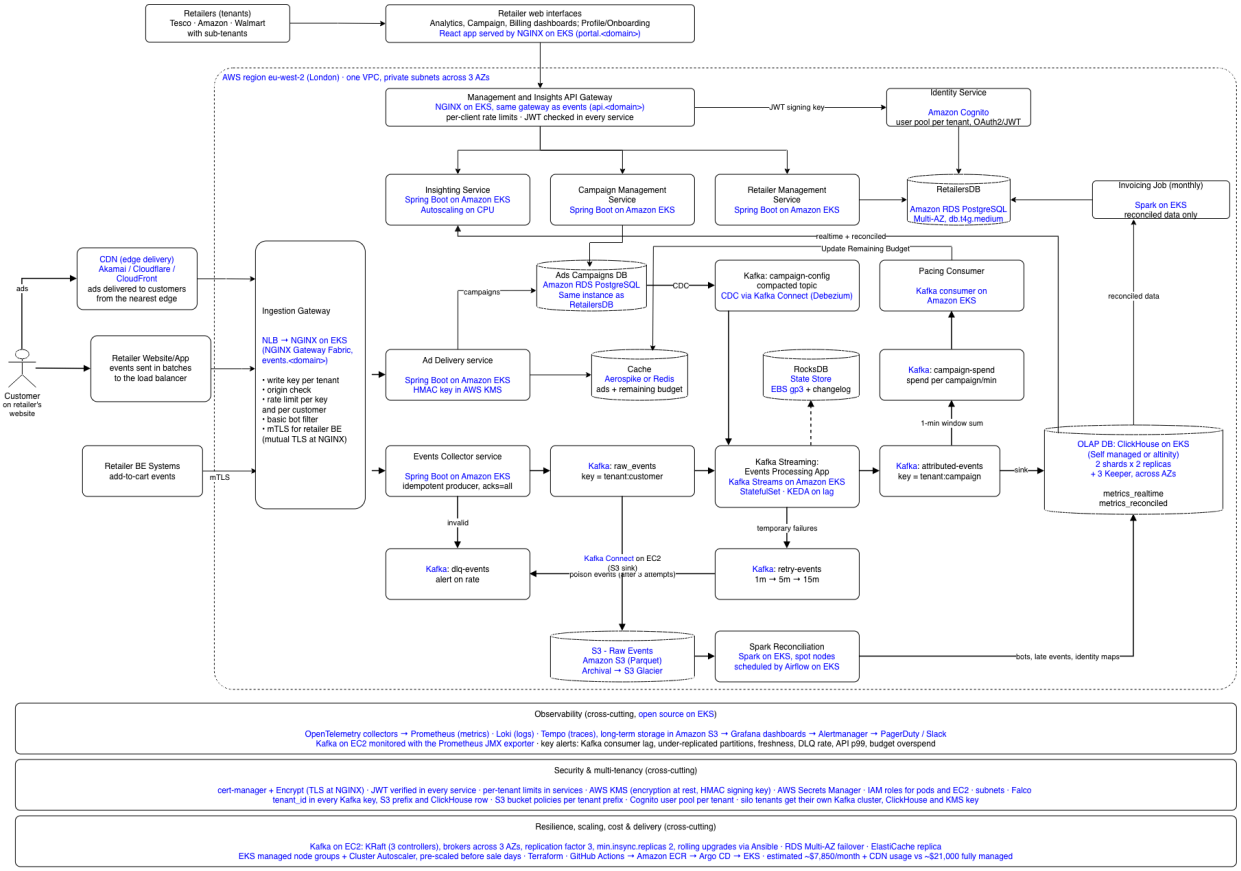
The deployment diagram follows the high-level design box for box, with the AWS service under each name. Each data-residency region (London for UK retailers) runs a full copy in one VPC across three availability zones, as the diagram at the end of this section shows.

Managed where failure is costly and the saving small (databases, cache, identity, S3, CDN); **self-run where the saving is large and the skills exist** (Kafka, ClickHouse, NGINX, Spark, observability). Estimated cost: about $7,850 a month plus CDN usage, against about $21,000 fully managed.

Component	Runs on	Managed / self-run
Ad delivery to customers	CDN (Akamai, Cloudflare or CloudFront); origin NGINX, then Ad Delivery	Managed
Portal, API Gateway, Ingestion Gateway	NLB → NGINX on EKS (NGINX Gateway Fabric), one hostname each; mTLS for retailer back ends	Self-run, portable
Identity Service	Amazon Cognito, user pool per tenant	Managed
Spring Boot services (Insighting, Campaign, Retailer, Ad Delivery, Events Collector)	Amazon EKS; HPA on CPU; HMAC key in AWS KMS	Managed control plane
RetailersDB, Ads Campaigns DB	One Amazon RDS PostgreSQL Multi-AZ instance, db.t4g.medium	Managed
Cache	Redis on Amazon ElastiCache (Aerospike if budget checks outgrow it)	Managed
Kafka and Kafka Connect	EC2, KRaft (3 controllers), brokers across 3 AZs; Debezium and S3 sink connectors	Self-run

Component	Runs on	Managed / self-run
Kafka Streams app, Pacing Consumer	EKS StatefulSet, RocksDB on EBS gp3, KEDA on consumer lag	Self-run library
ClickHouse	EKS via Altinity or official operator; 2 shards × 2 replicas, 3 Keepers	Self-run
S3 raw events	S3 Standard 90 days, then Glacier	Managed
Spark reconciliation, invoicing; Airflow	Spark on EKS, spot nodes; Airflow on EKS	Self-run
Platform and delivery	EKS managed node groups + Cluster Autoscaler; Terraform; GitHub Actions → ECR → Argo CD	Mixed

Edge and Regions. Ads go through the CDN, which serves cacheable content from the nearest edge and forwards per-customer decisions to the origin, while absorbing DDoS traffic. Event calls are unique and go straight to NGINX. The management portal has only a few thousand users and needs no CDN. A new region (for example, US East) is a copy of the stack; DNS and the CDN route customers to their nearest region, keeping each retailer's data in-region.



Cross-cutting concerns

Scalability

Rough sizing for a large retailer on a sale day(Peak Load), to be replaced with real traffic figures:

Input	Assumption	Result
Customers per day	20 million, 15 page views, 3 ads each	900 million impressions/day
Clicks	1.5% of impressions	13.5 million/day
Add-to-cart events	2 per customer	40 million/day
Average rate	~950 million events ÷ 86,400 s	~11,000 events/s
Peak	10× average during a sale	~110,000 events/s
Raw storage	~500 bytes per event, compressed ~5×	~95 GB/day in S3

Input	Assumption	Result
Click store	13.5 million clicks × 7 days × ~100 bytes	~10 GB across instances

- **Gateways and services** are stateless; Autoscaling adds pods based on CPU and request rate.
- **Kafka**: 100 partitions for raw_events (~1,000 events/s each at peak); more brokers add throughput. If processing falls behind, Kafka holds the backlog, and nothing is dropped.
- **Kafka Streams** scales to one instance per partition; KEDA autoscales on consumer lag.
- **ClickHouse** is sharded by tenant and replicated; reads spread across replicas.
- **Pre-scaling** before known peaks (Black Friday, Big Billion Days) avoids waiting for autoscaling.

Resilience and failure scenarios

Kafka uses replication factor 3 across three zones with min.insync.replicas=2; RDS and ElastiCache fail over automatically. If the real-time path fails completely, nothing is lost: S3 still receives every event and reconciliation rebuilds the numbers.

Scenario	What happens
Happy path	A Monday click and a Wednesday add-to-cart share tenant:customer, reach the same instance, and the click store produces an attributed basket.
Malformed event	Fails validation and goes straight to dlq-events with the reason; the DLQ rate is alerted on.
Temporary failure	A click for a campaign created seconds ago goes to retry-events and succeeds a minute later.
Poison message	Fails all three retries (1, 5, 15 minutes), lands in dlq-events, is fixed and replayed.
Duplicate event	A retried event with the same event_id is found in seen-event-ids and dropped.
Late event	An add-to-cart before its click waits 10 minutes; anything later is fixed by reconciliation.
Instance crash	The open transaction aborts; a warm standby takes the partitions and resumes from the last committed offset.
Budget exhausted	Below a 2% safety margin, Ad Delivery stops serving the campaign.

Scenario	What happens
Anonymous customer logs in	Same device and cookie, so the real-time join works; cross-device matches come from reconciliation.

Security and Multi-tenancy

- **Events:** per-tenant write keys, origin checks, rate limits, and HMAC-signed impression tokens so clicks can't be forged; add-to-cart events over mTLS.
- **Users:** OAuth2 JWTs from Cognito, verified in every service, with role checks and campaign ownership.
- **Data:** TLS everywhere (cert-manager and Let's Encrypt at NGINX), KMS encryption at rest, secrets in AWS Secrets Manager (Vault off AWS), IAM roles for pods, Falco for runtime threats; consent-aware tracking.
- **Isolation:** tenant_id in every Kafka key, S3 prefix and ClickHouse row, enforced by ClickHouse row policies and S3 IAM policies even if application code has a bug.
- **Fairness and configuration:** per-tenant Kafka quotas, gateway limits and ClickHouse query quotas. Each tenant's attribution and billing settings come from RetailersDB.
- **Pooled by default, silo on demand:** a large retailer can get dedicated topics, ClickHouse cluster and KMS keys from the same code.

Monitoring and alerting

OpenTelemetry collectors feed Prometheus (metrics; Kafka through the JMX exporter), Loki (logs with tenant_id and event_id) and Tempo (traces), with Grafana dashboards and Alertmanager to PagerDuty and Slack. SLOs: 99.9% ingestion availability, freshness under 60 s and API p99 under 500 ms, alerting on error-budget burn.

Layer	Key metrics	Example alert
Ingestion	Events/s per tenant, rejected %, p99 latency	Rejections above 1% for a tenant
Kafka and streams	Consumer lag, under-replicated partitions, freshness, DLQ rate	Lag growing for 5 minutes
ClickHouse and API	Query p95/p99, 5xx rate, throttled requests	API p99 above 500 ms
Business	Reconciliation delta, budget overspend, traffic per tenant	Delta above 5%; a tenant's traffic drops to zero

Cost

Raw history lives in S3 and Glacier, not Kafka or ClickHouse; ClickHouse stores aggregates only. Compute scales down after peaks and Spark runs on spot nodes. Resources are tagged by tenant for a daily cost-per-tenant report, with an alert on unexpected jumps.

Trade-offs and challenges

Performance vs accuracy. Two paths run over the same events: a fast one with preliminary numbers within seconds, and an accurate one with final, auditable numbers the next morning. The API labels which is which, and billing uses only the accurate path, so dashboards stay fast, and no invoice uses a preliminary number.

Decision	Chosen	Alternative	Why
Stream framework	Kafka Streams	Apache Flink	No separate cluster; Flink wins for very large state or complex event-time logic
Analytics store	ClickHouse	Druid, Pinot, Elasticsearch	Mergeable exact unique counts, SQL, simpler to run
Join state	Local RocksDB	Redis or Cassandra per event	No network call per event; costs restore time after a crash
Raw storage	S3 Parquet	HBase, Cassandra	Far cheaper; no single-row lookups needed
Tenancy	Pooled, silo on demand	Silo for all	Cheap for many small retailers; isolation still available
Attribution	Last click, configurable	Multi-touch	Simple and explainable; over-credits the last ad

- **Identity:** customers who don't log in or reject cookies can't be linked across devices in real time; reconciliation recovers some matches.
- **Fraud and bots:** simple rules at the edge; pattern detection runs in the nightly batch.
- **Schema changes** across many retailer integrations: a schema registry with backwards-compatible versions.
- **Extending:** a new event type needs a new event_type and aggregate table; view-through attribution reuses the same join; a new region is a copy of the deployment.